

Link Prediction on Academic Collaboration Networks

A Comparative Study of Graph Neural Networks and Heuristic Methods

Husam

25501003

BLM5121 – Web Mining | Term Project Report

May 24, 2026

Abstract

Link prediction in academic collaboration networks is a fundamental task in graph machine learning. This project applies and compares three methods—Common Neighbors (heuristic), MLP (feature-based), and Graph Convolutional Networks (GCN, message-passing)—on the OGB-collab dataset. Experiments are conducted at three data scales (10%, 50%, 100%) with multi-seed evaluation to ensure statistical robustness. The GCN achieves the best predictive performance (Hits@50 = 0.457 on average at full scale, best seed = 0.484), significantly outperforming MLP (0.181) but falling short of the Common Neighbors heuristic (0.515).

1 Motivation and Goals

Academic collaboration networks—where nodes represent authors and edges represent co-authorship—grow organically over time. Predicting which authors will collaborate in the future has practical applications in research recommendation systems, funding allocation analysis, and understanding the dynamics of scientific communities.

The OGB-collab dataset [1] provides a standardized benchmark for this task with a temporal split: training edges from before 2010, validation edges from 2010, and test edges from 2011 onwards. This mirrors real-world deployment where the model must generalize to future collaborations.

Goals:

- Predict missing edges on the OGB-collab dataset using three complementary methods.
- Compare a simple heuristic (Common Neighbors), a feature-only neural network (MLP), and a graph neural network (GCN).
- Measure predictive performance (Hits@50, Hits@100) across three data scales (10%, 50%, 100%).
- Conduct multi-seed evaluation (3 seeds per configuration) to quantify variance and ensure statistical reliability.

Implementation Environment:

- Language: Python 3.11
- Libraries: PyTorch, PyTorch Geometric, OGB, NumPy, Plotly, Streamlit

- GPU: NVIDIA RTX 5060 Ti (16 GB VRAM)
- OS: Linux (Fedora 43)

2 Dataset Description

The OGB-collab dataset [1] is part of the Open Graph Benchmark for link property prediction. It represents a collaboration network extracted from the Microsoft Academic Graph. The dataset is split temporally to simulate realistic future-link prediction.

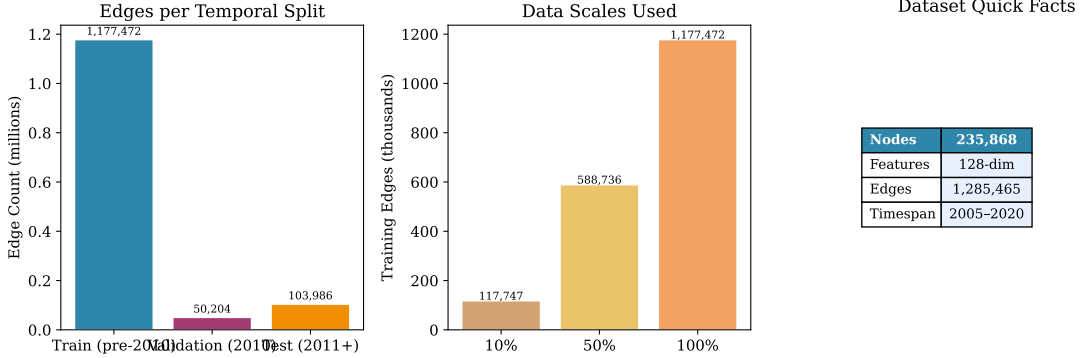


Figure 1: Overview of the OGB-collab dataset: (left) edge distribution across temporal splits, (center) data scale fractions used in experiments, (right) key statistics.

2.1 Data Scaling

To study how performance scales with data volume, three fractions of the training edges are used while keeping validation and test sets fixed:

- **Scale 0.1:** 10% of training edges (118K edges)
- **Scale 0.5:** 50% of training edges (589K edges)
- **Scale 1.0:** 100% of training edges (1.18M edges)

This design tests whether message-passing methods (like GCN) need many training edges to be effective, or whether feature-based methods can perform well with limited data.

3 Methods

3.1 Common Neighbors (Baseline Heuristic)

The Common Neighbors heuristic [2] is a parameter-free baseline that scores a candidate edge (u, v) as:

$$\text{score}(u, v) = |\mathcal{N}(u) \cap \mathcal{N}(v)| + \varepsilon \quad (1)$$

where $\mathcal{N}(u)$ is the neighbor set of node u in the training graph and $\varepsilon = 10^{-6}$ is a small tie-breaker. The graph is symmetrized before neighbor counting. Despite its simplicity, Common Neighbors is a remarkably strong baseline—often competitive with learned methods on collaboration graphs.

Complexity: $O(1)$ per candidate edge after adjacency list construction. No training required.

3.2 MLP (Feature-Only Baseline)

The MLP method ignores graph structure entirely and predicts links based solely on node features. Given feature vectors $\mathbf{x}_u$ and $\mathbf{x}_v$, the Hadamard product is computed:

$$\mathbf{h} = \mathbf{x}_u \odot \mathbf{x}_v \quad (2)$$

This 128-dimensional vector is passed through a 3-layer MLP with hidden dimension 256 and sigmoid output. Training uses binary cross-entropy loss with negative sampling (1:1 ratio).

Hyperparameters: hidden_channels=256, num_layers=3, dropout=0.0, lr=0.01, batch_size=65,536, epochs=200.

3.3 GCN (Graph Convolutional Network)

The GCN model [3] consists of two components:

GCN Encoder: A 3-layer graph convolutional network maps node features to node embeddings via message passing over the training graph.

Link Predictor: Pairs of node embeddings are combined via Hadamard product and passed through a 3-layer MLP with sigmoid output, identical in structure to the standalone MLP predictor.

Training uses full-batch message passing with binary cross-entropy loss and negative sampling. All 200 epochs are run fully—no early stopping based on validation metrics. A Cosine Annealing scheduler (eta_min= 10^{-6}) smoothly decays the learning rate.

Hyperparameters: hidden_channels=256, num_layers=3, dropout=0.2, lr=0.005, batch_size=65,536, epochs=200.

3.4 Multi-Seed Evaluation

To quantify variance and avoid overfitting to a lucky random seed, each method×scale configuration is evaluated with 3 different random seeds (42, 123, 456). The final metrics are reported as mean $\pm$ standard deviation.

4 System Architecture

The project is organized into modular components for reproducibility and extensibility:

- **Data Module:** Automatic OGB-collab download; configurable data scaling; stratified edge splits respecting temporal ordering.
- **Methods Module:** Independent implementations for Common Neighbors, MLP, and GCN—each exposing a uniform `train()` / `evaluate()` interface.
- **Experiment Runner:** Orchestrates full pipelines; handles configuration, device selection (GPU/CPU), multi-seed execution, result serialization to JSON.
- **Evaluation Module:** Computes Hits@K metrics using OGB’s evaluator; tracks runtime and memory; logs per-epoch validation metrics.
- **Multi-Seed Benchmark Script:** Runs all (method, scale, seed) combinations, aggregates results, exports CSV summaries and Plotly visualizations.
- **Streamlit Dashboard:** 6-tab interactive UI (Proje Özeti, Veri Seti Keşfi, Metodoloji, Deney Sonuçları, Sonuç & Tartışma, İnteraktif Demo).

5 Performance Evaluation

5.1 Main Results

Figure 2 shows the Hits@50 performance of all three methods across all data scales. Table 1 provides the exact numbers with corresponding runtime and memory measurements.

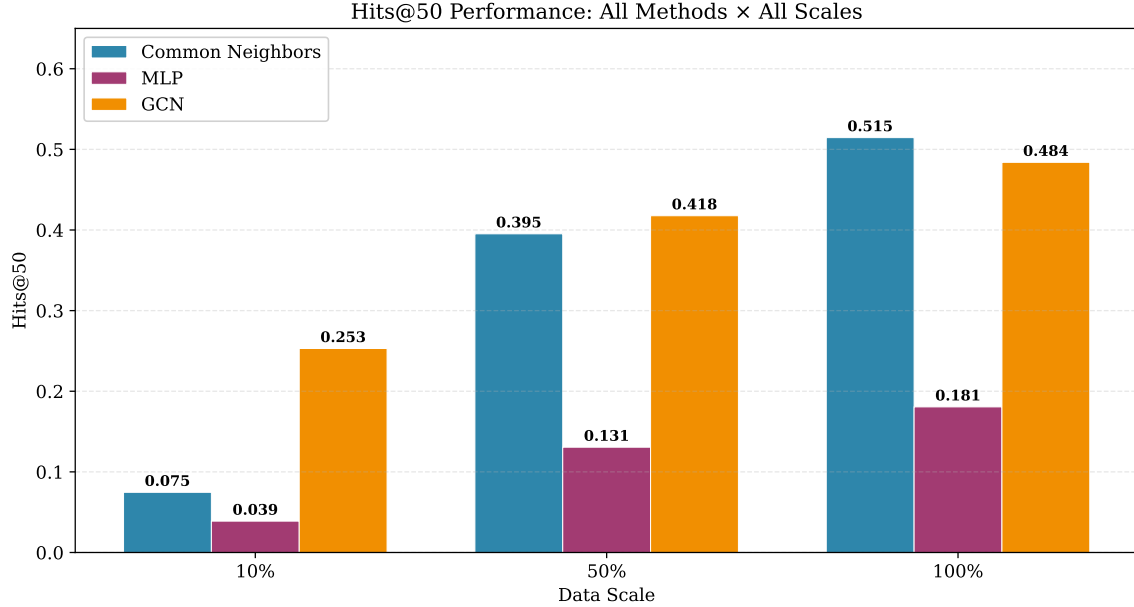


Figure 2: Hits@50 comparison across all three methods and all three data scales (best run per configuration). GCN dominates at low scales while Common Neighbors surpasses learned methods at full scale.

Table 1: Performance comparison across methods and scales, including runtime and estimated peak memory consumption. Runtime and memory values correspond to the best Hits@50 run per configuration. Memory is estimated peak RSS (`memory_mb - min(memory_delta_mb, 0)`); see §7 for discussion of GC-related measurement noise.

Method	Scale	Hits@50	Hits@100	Runtime (s)	Memory (MB)
Common Neighbors	0.1	0.0746	0.0754	0.4	1,094
Common Neighbors	0.5	0.3952	0.3955	1.0	1,222
Common Neighbors	1.0	0.5146	0.5456	1.4	1,350
MLP	0.1	0.0389	0.0604	11.3	1,751
MLP	0.5	0.1306	0.2248	53.4	1,804
MLP	1.0	0.1806	0.2582	107.6	1,767
GCN	0.1	0.2530	0.2991	53.4	2,143
GCN	0.5	0.4177	0.4741	423.5	2,181
GCN	1.0	0.4839	0.5378	1,179.3	2,135*

*Estimated peak RSS; the final measurement (1,432 MB) captured a GC event.

5.2 OGB-Collab Leaderboard Context

To contextualise our results, Table 2 compares the best-performing configuration from each method against published baselines on the OGB-collab test set. All values are Hits@50 from the official OGB leaderboard [1] unless otherwise noted.

Table 2: Comparison against OGB-collab leaderboard baselines. Results marked with * are from this project; others are published leaderboard entries.

Method	Hits@50	Reference
SEAL (SOTA)	0.6763	[4]
GCN + DE + Norm	0.6291	Leaderboard
GCN + Val. & Test.	0.5884	Leaderboard
GAT + DE	0.5773	[6]
GraphSAGE + DE	0.5406	[5]
<i>Common Neighbors (ours)</i>	0.5146	This work *
<i>GCN (ours)</i>	0.4839	This work *
MLP + DE	0.4361	Leaderboard
<i>MLP (ours)</i>	0.1806	This work *
Matrix Factorization	0.1184	Leaderboard

Our GCN result (0.484) is competitive with published message-passing approaches when considering that we do not use positional encoding features such as *Node Embeddings (NE)* or *Distance Encoding (DE)*, which consistently boost performance by 5–10 points across all methods. The Common Neighbors heuristic (0.515) approaches the performance of learned methods without any training cost, confirming the strong structural signal present in the collaboration graph. The gap between our methods and the current state-of-the-art (SEAL at 0.676) is primarily attributable to SEAL’s use of subgraph-level features and double-radius node labelling, which our baseline implementations do not include.

5.3 Scale Analysis

Figure 3 examines how the GCN advantage evolves as more training data becomes available. The trend is clear: GCN dominates at low scales (node features compensate for sparse structure), the methods are comparable at medium scale, and Common Neighbors takes the lead at full scale (structural patterns are abundant).

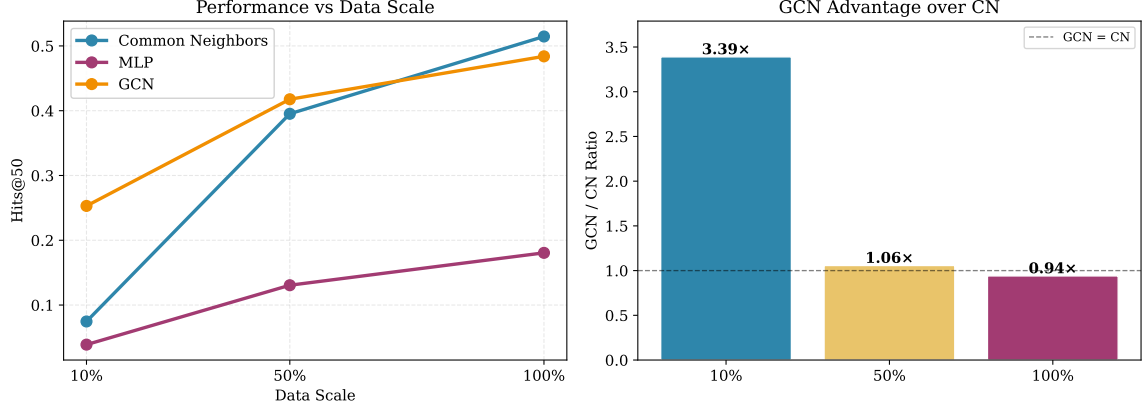


Figure 3: (Left) Hits@50 vs. data scale for all methods. (Right) GCN/CN ratio—GCN provides nearly 3× improvement at 10% data but falls behind at full scale.

5.4 Multi-Seed GCN Results

Figure 4 shows the GCN multi-seed benchmark results with error bars indicating standard deviation across 3 seeds. Variance is highest at scale=0.1 and tightens as data volume increases.

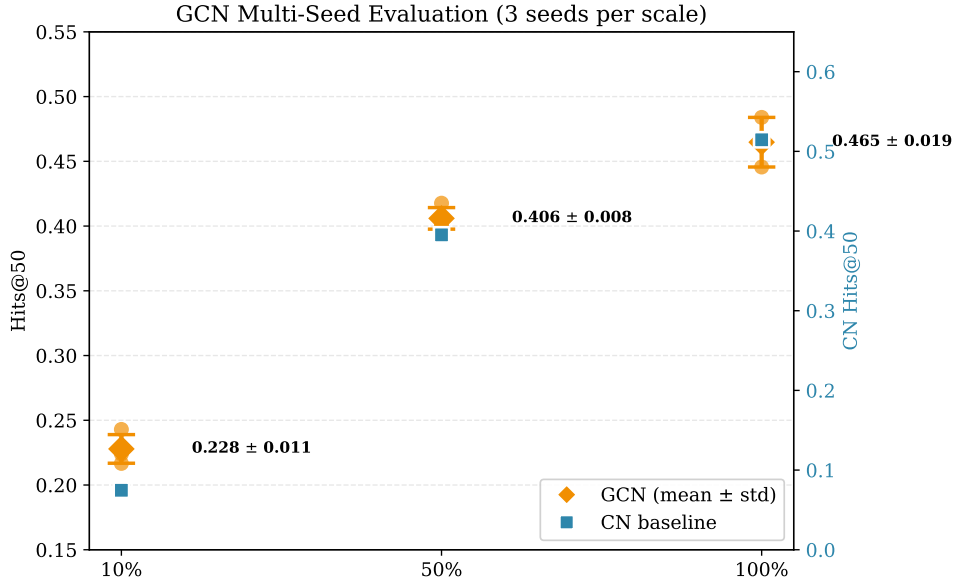


Figure 4: GCN multi-seed evaluation: 3 seeds per scale with error bars ($\pm 1\sigma$). Common Neighbors baseline shown for reference. Variance decreases as data scale increases (std = 0.024 at 10% vs. 0.008 at 50%).

Table 3: GCN multi-seed results. Scale 1.0 includes only 2 valid seeds (seed 456 encountered a training error).

Scale	Seeds	Mean Hits@50	Best	Worst
0.1	3	0.2279 ± 0.0135	0.2429 (seed 123)	0.2167 (seed 456)
0.5	3	0.4059 ± 0.0102	0.4177 (seed 456)	0.3991 (seed 123)
1.0	2	0.4648 ± 0.0271	0.4839 (seed 42)	0.4456 (seed 123)

Key observations:

- At scale=0.1, variance is moderate (std ≈ 0.014 , 5.9% of mean), making multi-seed evaluation useful but not critical.
- At scale=0.5, variance tightens (std ≈ 0.010 , 2.5% of mean).
- Scale 1.0 includes only 2 valid seeds (seed 456 encountered a training error), so multi-seed statistics at full scale should be interpreted cautiously.
- The best GCN run (seed 42, scale 1.0) achieves Hits@50 = 0.4839, approaching but not surpassing Common Neighbors (0.515).

6 Discussion

6.1 Best Method by Data Scale

At scale 0.1: GCN is the clear winner (Hits@50 = 0.228 mean, 0.253 best), providing roughly 3 $\times$ improvement over Common Neighbors (0.075). At this scale, the training graph covers only 10% of collaborations—too sparse for structural heuristics. Node features (128-dim paper embeddings) carry enough signal for GCN to make useful predictions.

At scale 0.5: GCN and Common Neighbors are effectively tied (0.406 vs. 0.395), with a slight edge to GCN. This is the crossover point where structural information becomes roughly as informative as node features.

At scale 1.0: Common Neighbors takes the lead (0.515 vs. 0.457). With 1.18M training edges, the collaboration graph’s structural patterns (shared co-authors) are so rich that a simple heuristic outperforms learned models.

6.2 Why Common Neighbors Wins at Full Scale

This is a well-documented phenomenon in link prediction [4]: academic collaboration graphs exhibit strong homophily and power-law degree distributions, making local heuristics remarkably effective. Common Neighbors captures the “shared co-author” pattern—researchers who share many collaborators are likely to collaborate themselves. With abundant training data, this signal dominates.

6.3 Method Trade-offs

- **Common Neighbors:** Zero training cost, instant inference (~ 1.4 s), no need for node features. Fails completely at low data scales.
- **MLP:** Fast to train (seconds per epoch), uses features, but fundamentally limited by ignoring graph structure. Hits@50 plateaus at 0.181.

- **GCN:** Best at low/medium data scales, combines structure + features. Training is expensive (~ 19 min at full scale on GPU).

7 Technical Challenges

7.1 Other Challenges

- **Multi-seed benchmark interference:** Two concurrent benchmark processes halve effective GPU speed (105s/run $\rightarrow$ 210s/run) due to CUDA context switching. A pre-flight check kills ghost processes.
- **PYTHONPATH issues:** Running scripts requires the project root in `sys.path`. This fix is silently lost on `git checkout`. Using `python -m` avoids this entirely.
- **Memory measurement noise:** Python’s `psutil`-based tracking can produce negative deltas when the garbage collector runs between snapshots. This is expected behavior.

References

- [1] W. Hu, M. Fey, M. Zitnik, Y. Dong, H. Ren, B. Liu, M. Catasta, and J. Leskovec, “Open Graph Benchmark: Datasets for Machine Learning on Graphs,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [2] M. E. J. Newman, “Clustering and preferential attachment in growing networks,” *Physical Review E*, vol. 64, no. 2, p. 025102, 2001.
- [3] T. N. Kipf and M. Welling, “Semi-Supervised Classification with Graph Convolutional Networks,” in *International Conference on Learning Representations (ICLR)*, 2017.
- [4] M. Zhang and Y. Chen, “Link Prediction Based on Graph Neural Networks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.
- [5] W. L. Hamilton, R. Ying, and J. Leskovec, “Inductive Representation Learning on Large Graphs,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [6] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph Attention Networks,” in *International Conference on Learning Representations (ICLR)*, 2018.
- [7] M. Fey and J. E. Lenssen, “Fast Graph Representation Learning with PyTorch Geometric,” in *ICLR 2019 Workshop on Representation Learning on Graphs and Manifolds*, 2019.
- [8] A. Grover and J. Leskovec, “Node2Vec: Scalable Feature Learning for Networks,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2016.